Vortex Gaussians: Real-Time Simulation-Driven Fire and Smoke as Native Gaussian-Splatting Content

Anonymous Author(s)

Abstract

Fire and smoke remain absent from Gaussian-splatting (GS) scenes: physics-integrated GS handles solids and liquids, GS fire methods only reconstruct observed flames, and the one system that simulates new fire composites ray-marched volumes over an opaque backdrop at seconds per frame. We present the first system that generates fire and smoke by forward physical simulation in real time and renders them as *native* Gaussian-splatting content. Our key insight is that Lagrangian vortex particles and 3D Gaussians are the same class of point primitive: one particle set serves simultaneously as simulation state and render primitive—position to Gaussian mean, temperature to emissive blackbody color, density to opacity, velocity gradient to anisotropic covariance—with no voxelization, meshing, or simulation-to-renderer conversion anywhere: flame, smoke and embers are drawn by one shader with a single Gaussian footprint, the same one used for the scene's splats. A two-level layered vortex-in-cell solver in vorticity–streamfunction form drives the particles; one depth-sorted emission–absorption rasterizer composites flame, smoke, embers, and reconstructed 3DGS scenes with per-primitive mutual occlusion, so a foreground object correctly occludes the flame. Combustion state further lives directly on the scene's splats: the flame ignites the reconstruction, fire spreads and chars in real time, and burning geometry re-seeds the solver—a bidirectional simulation–scene loop. Our single-file WebGL2 prototype runs the default configuration (11.7k flame Gaussians) at 24 ms per frame (41 FPS at 1600×1250, simulation single-threaded in CPU JavaScript), scales linearly in slice count, and ships a deterministic-replay harness that makes every reported number bit-reproducible. We also measure what the layered approximation costs: against a matched 3D reference its in-plane spectrum is markedly steeper, which we report as a scoping result rather than a fidelity claim.

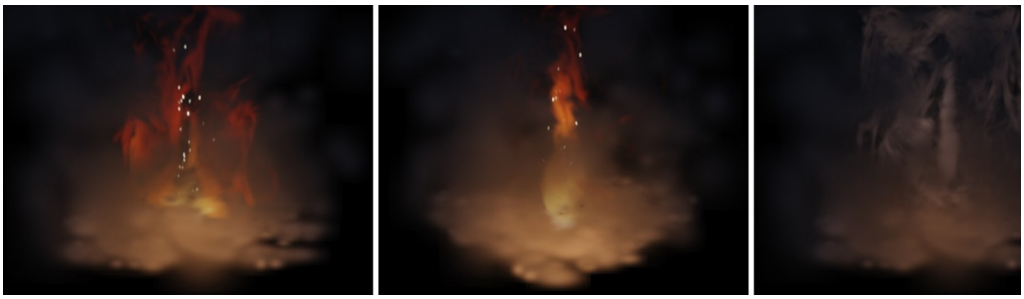

Figure 1. *Real-time, simulation-driven fire and smoke rendered as native Gaussian-splatting content. Left: a flame driven by nine layered 2D vortex-in-cell simulations, in which every vortex particle is an emissive anisotropic Gaussian (white-hot core → orange → dark-red tips, with embers). Middle: the same fire from an orbiting camera; view-aligned slices and per-primitive depth sorting give correct parallax for the rendering camera against a dark 3DGS background scene that the fire locally illuminates. Right: the identical particle set re-shaded as smoke (absorption-dominated compositing with warm underlighting). All results run in real time in a browser.*

1 Introduction

Radiance-field capture has made it routine to bring real environments into the computer as 3D Gaussian-splatting (3DGS) scenes that render photorealistically in real time (Kerbl et al. 2023). A growing body of work now asks how such scenes can be brought to *life*: physics-integrated Gaussian methods animate elastic solids and liquids by attaching continuum solvers to the reconstructed primitives (Xie et al. 2024; Feng et al. 2025; Zhong et al. 2024; Jiang et al. 2024). Yet the most visually iconic dynamic phenomena—fire and smoke—remain conspicuously missing from this ecosystem. Existing Gaussian representations of flames are *inverse*: they reconstruct an observed fire from video (Nazarenus et al. 2025; Shui et al. 2024) and can only replay it, as do neural volumetric methods for smoke and fluids (Chu et al. 2022; Yu et al. 2023; Gao et al. 2025). The one system that synthesizes new fire in reconstructed GS scenes, FieryGS (Shen et al. 2026), runs an offline 256^3 Eulerian solver and ray-marches the resulting volume over the GS scene treated as an opaque backdrop, at seconds per frame. We do not claim a like-for-like speedup over it—its 256^3 grid carries three orders of magnitude more degrees of freedom than our slice stack, and we have not run it—but the gap in *operating regime*, offline versus interactive, is what motivates a differently-shaped design.

This gap is not accidental. Gaseous combustion has neither a surface to mesh nor persistent material points to track, so the standard physics-to-GS recipes (bind Gaussians to a deformation field; skin them to simulation particles) do not directly apply, and volumetric emission seems to demand ray marching.

Our starting point is an observation about *representation*: a Lagrangian vortex particle and a 3D Gaussian are the same kind of object. Both are unordered point primitives carrying a position and a handful of per-point attributes; both live in flat GPU buffers; and both are consumed by a sort-and-blend operator (Biot–Savart summation over particles; front-to-back alpha compositing over splats). We exploit this isomorphism literally: **one particle set is simultaneously the simulation state and the render primitive.** Each vortex particle carries circulation Γ, temperature T, and density d; the renderer reads the very same particle as an emissive anisotropic Gaussian whose mean is the particle position, whose color is a blackbody function of T, whose opacity follows d, and whose covariance is evolved by the local velocity gradient. No voxelization, no marching cubes, no simulation-to-renderer resampling of any kind separates physics from pixels. In the spirit of PhysGaussian's "what you see is what you simulate" (Xie et al. 2024), our principle is *what you simulate is what you splat*.

For dynamics we revisit a production idea. Industrial Light & Magic's film fire system (Horvath and Geiger 2009) showed that a stack of independent, camera-facing 2D slice simulations, driven by a coarse directable 3D pass, is perceptually sufficient for even hero-shot fire—but it ran on a GPU farm at ~ 20 s per slice per frame. We make this two-level architecture real-time and fully Lagrangian: each slice runs a 2D *vortex-in-cell* (VIC) solver in which buoyancy injects circulation into particles through the baroclinic term $\beta \partial T / \partial x$, a small streamfunction Poisson solve reconstructs a divergence-free velocity at $O(N_p + G)$ cost (replacing $O(N_p^2)$ Biot–Savart summation), and a coarse global solver nudges all slices toward one coherent bulk motion. Because the slices' particles are already Gaussians, the entire stack—flame, smoke, embers, and the static background scene—feeds a single depth-sorted emission–absorption rasterizer with

HDR blackbody shading, so the fire correctly occludes, is occluded by, and locally illuminates the environment.

We validate the system with an interactive prototype implemented as a single-file WebGL2 application whose simulation runs in JavaScript on the CPU, demonstrating that the method is cheap enough for real time even without GPU compute. We further contribute a decoupled per-layer interface that hot-swaps each slice between the physical solver and a procedural curl-noise source at runtime, which we use both as a development methodology and as an ablation instrument.

In summary, our contributions are:

- **A real-time combustion loop between a forward flame simulation and the splats of a captured scene.** Combustion state—heat, fuel, char, glow—lives *directly on the scene's own Gaussians*: the simulated flame ignites the reconstruction, fire spreads splat-to-splat, charring is written into the splats' rendered attributes, and burning geometry re-seeds the solver with new plumes (§5.6). The spread physics itself is conventional (Pirk et al. 2017; Hädrich et al. 2021); what is new is the substrate and the closed loop—because burning geometry feeds the solver rather than only the renderer, ignition can propagate through the simulated flow and not only along the adjacency graph.

- **GS-native composition that makes the loop possible.** An asymmetric exact/bucketed two-stream depth interleave merges the fire and a loaded reconstruction into one global depth order at no measured overhead, giving per-primitive mutual occlusion and in-scene ignition (§5.5)—where prior art composites fire *over* GS scenes as an opaque backdrop (Shen et al. 2026). Fire, smoke, embers and scene splats share one shader, one Gaussian footprint, and one emission–absorption blend.

- **Particle–Gaussian unification for gaseous media.** One particle set is simultaneously simulation state and render primitive, under the mapping $(x, T, d, u, \nabla u) \mapsto (\mu, c, \alpha, \Sigma)$ with zero conversion passes—demonstrated across flame, smoke, and embers (§3, §5). Gases have neither a rest configuration nor persistent material points, which is precisely what *vorticity-carrying* particles supply; correspondingly, covariance is evolved incrementally by the velocity gradient, $\dot{\Sigma} = L\Sigma + \Sigma L^{\top}$, with clamps on the rendered quantity—the rest-configuration-free counterpart of covariance-from-deformation (Xie et al. 2024).

- **A solver cheap enough to close the loop at interactive rates, and an honest account of what it costs.** Both levels are in vorticity–streamfunction form and need no pressure solve, which is what buys nine coupled slices plus a coarse solve in a CPU frame budget (§4). We claim this as an engineering enabler, not a fluids advance—the components are classical—and we measure the price of the layered approximation against a matched 3D reference (§7).

We quantify the design space—per-frame cost scaling in slices, particles, and scene splats; VIC vs. naive Biot–Savart; slice orientation; covariance dynamics; and, centrally, the layered approximation itself, measured as energy spectra and cross-plane coherence against a matched true-3D Boussinesq reference—through a deterministic-replay harness that makes every number bit-reproducible from a seed (§7).

2 Related Work

2.1 Fire and Smoke Simulation

Visually plausible gaseous simulation in graphics descends from the unconditionally stable semi-Lagrangian advection of Stam (1999), extended with vorticity confinement to fight numerical dissipation (Fedkiw et al. 2001; Steinhoff and Underhill 1994) and specialized to combustion by Nguyen et al. (2002), whose separate fuel/hot-product phases and blackbody radiance remain the canonical physically based fire model. Heat-threshold fire *spread* over solid substrates—fuel, ignition temperatures, charring—is likewise established, on botanical meshes (Pirk et al. 2017) and at wildfire mesoscale (Hädrich et al. 2021); we adopt this conventional model but carry its state on the splats of a captured scene (§5.6). Real-time grid solvers moved to the GPU early (Harris 2004; Crane et al. 2007).

Vortex methods discretize the vorticity form of the Euler equations on Lagrangian elements—particles (Chorin 1973; Cottet and Koumoutsakos 2000; Park and Kim 2005; Selle et al. 2005), filaments (Angelidis and Neyret 2005; Weißmann and Pinkall 2010), or sheets (Pfaff et al. 2012; Brochu et al. 2012) —which auto-satisfy incompressibility in reconstruction, need no pressure projection, and preserve turbulent detail with striking economy. Their classical bottleneck is velocity recovery: naive Biot–Savart summation is $O(N_p^2)$, mitigated by treecodes and fast multipole methods (Brochu et al. 2012) or by particle–grid hybrids (vortex-in-cell) (Cottet and Koumoutsakos 2000). Vorticity-based solvers continue to advance, most recently through flow-map formulations (Wang et al. 2024). We adopt a vortex-in-cell formulation per slice, obtaining $O(N_p + G)$ velocity reconstruction on grids small enough for JavaScript, and we inherit the Lagrangian virtue that matters most here: *the solver's degrees of freedom are already renderable points*. Brochu et al. (2012) argued that simulation cost should scale with rendered detail and named real-time vortex-based fire an open problem; our system is one answer.

Production layered fire. Horvath and Geiger (2009) decompose film fire into a coarse, directable 3D particle pass (a PIC/FLIP variant (Zhu and Bridson 2005)) and a stack of independent, camera-facing 2D Navier–Stokes slice simulations (typically $\sim 2048^2$; film examples at 2048×1556 with 32–128 slices), composited near-to-far with Planck blackbody shading. The insight that fine detail perpendicular to the view plane is perceptually invisible justifies 2D refinement. Their slices never exchange simulation state with one another; coherence comes from the shared coarse forcing, splatted with a Gaussian depth kernel spanning ~ 8 slices. The system is offline (~ 20 s per slice per frame on a GPU farm), renders slices as textures composited against holdout mattes, and its stated limitations—fast-moving cameras, flows perpendicular to the view plane, no inter-slice communication—map directly onto the design decisions we revisit. We replace per-slice grid Navier–Stokes with 2D vortex particles whose Lagrangian state survives slice re-orientation, replace slice textures with Gaussians shared with

the environment, and replace the fixed projection kernel with a coarse global *solver* whose influence is a continuously tunable velocity blend.

Layered and billboard volume rendering. Half-angle slicing, volumetric billboards, and spherical billboards render volumes as stacks of blended proxies (Decaudin and Neyret 2009; Umenhoffer et al. 2006); games have long shipped 2.5D flipbook and card-based fire. These techniques require the three ingredients our renderer gets for free from splatting: depth-sorted blending, parallax from depth offsets, and camera-facing orientation.

2.2 Gaussian Splatting

3DGS (Kerbl et al. 2023) represents radiance fields as anisotropic Gaussians rasterized by tile-based, depth-sorted front-to-back alpha blending, following the EWA volume-splatting lineage (Zwicker et al. 2001); anti-aliased variants (Yu et al. 2024) and physically based volumetric-primitive transport (Condor et al. 2025) refine the formulation. Condor et al. (2025) in particular show Gaussian mixtures are a sound basis for *emissive* media under ray tracing; we target the rasterized, real-time end of that trade-off. Dynamic-scene extensions obtain motion from data—per-frame optimization (Luiten et al. 2024) or learned deformation fields (Wu et al. 2024)—and thus replay observed dynamics rather than generate new ones.

Physics-integrated Gaussians animate reconstructed scenes forward in time: PhysGaussian evolves Gaussian kernels under MPM continuum mechanics (Xie et al. 2024), Gaussian Splashing couples position-based fluids with splat rendering (Feng et al. 2025), Spring-Gaus embeds spring-mass systems (Zhong et al. 2024), and VR-GS makes such coupling interactive (Jiang et al. 2024). All target solids or liquids—media with material points or surfaces—and none address optically thin emissive volumes. Orthogonally, Gaussian Fluids (Xing et al. 2025) uses Gaussians as *basis functions* for the velocity field itself, solving incompressible flow by per-step optimization at 38–228 s per step, without any rendering; we use Gaussians as *render primitives* carried by a classical real-time solver. The two works bracket our claim from opposite sides: Gaussians suit both fluid simulation state and fluid appearance, and our contribution is to make one particle set serve both at once.

Gaussians for fire. FlameGS (Shui et al. 2024) and Gaussians on Fire (Nazarenus et al. 2025) reconstruct emissive flame light fields from images; PINF (Chu et al. 2022), HyFluid (Yu et al. 2023), FluidNexus (Gao et al. 2025), and Vid2Fluid (Zhao et al. 2025) recover smoke or fluid dynamics from video with physics priors. These are inverse problems. FieryGS (Shen et al. 2026) is the closest prior system and the only one that *synthesizes* fire in GS scenes: an MLLM assigns combustion properties to a reconstructed scene, a 256^3 Eulerian solver simulates fire, and a ray-marched volume is composited over the GS scene attenuated as an opaque background, at 2.37 s per frame (0.84 s of which is the fire/smoke ray march). Our system differs on every axis (Table 1): Lagrangian rather than Eulerian simulation, per-primitive depth interleaving rather than backdrop compositing, rasterization rather than ray marching, and real-time rather than offline. FieryGS's orthogonal capabilities—material inference, charring—are complementary to our pipeline.

Table 1. *Positioning. "GS-native" means the phenomenon is rendered as Gaussian primitives depth-interleaved with the scene, not composited over it.*

	gaseous emissive	forward physics	real time	GS- native	scene composed
Horvath and Geiger (2009)	✓	✓	—	—	matte
PhysGaussian (Xie et al. 2024)	—	✓	—	✓	✓
G. Splashing (Feng et al. 2025)	—	✓	—	✓	✓
G. Fluids (Xing et al. 2025)	—	✓	—	—	—
Fire recon. (Nazarenus et al. 2025)	✓	—	(render)	✓	—
FieryGS (Shen et al. 2026)	✓	✓	—	—	backdrop
Ours	✓	✓	✓	✓	✓

3 Overview

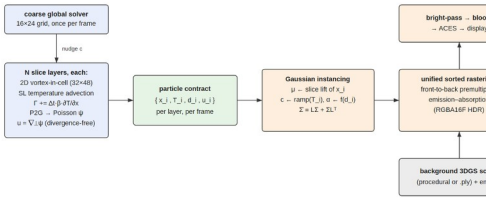

Figure 2. *System overview. A coarse global vortex solver drives N per-slice 2D vortex-in-cell simulations (left, §4). Each slice exposes its particles through a minimal contract (center); the renderer lifts each particle directly into an emissive anisotropic Gaussian (§5) and rasterizes all Gaussians—fire, smoke, embers, and the background 3DGS scene—in one depth-sorted emission–absorption pass with HDR post-processing. There is no voxelization or resampling anywhere in the pipeline.*

Figure 2 summarizes the system. The domain is a stack of N parallel slice layers (default $N = 9$) spanning the fire volume. Once per frame we advance a coarse global solver (§4.2) and then each layer's 2D vortex-in-cell simulation (§4.1); every layer exposes its particles through the contract

$$Layer.step(\Delta t) \rightarrow \{x_i \in \mathbb{R}^2, T_i, d_i, u_i \in \mathbb{R}^2\},$$

i.e., in-plane position, temperature, density, and velocity. The renderer is the *only* consumer of this contract: it lifts each particle into world space using the layer's slice frame (§4.3) and instances one emissive Gaussian per particle (§5). Because the contract is source-agnostic, any layer can be hot-swapped at runtime between the physical solver and a cheap procedural curl-noise plume (Bridson et al. 2007)—a property we exploit for incremental system verification and for ablations (§7).

4 Two-Level Layered Vortex Simulation

4.1 Per-Slice Vortex-in-Cell Dynamics

Each slice simulates buoyant flow in the vorticity–streamfunction form of the incompressible Euler equations. In 2D, vorticity is the scalar $\omega = \partial_x u_y - \partial_y u_x$ and the vortex-stretching term vanishes identically, leaving

$$\frac{\partial \omega}{\partial t} + (u \cdot \nabla)\omega = (\nabla \times f_b)_z = \beta \frac{\partial T}{\partial x}, \quad (1)$$

where the right-hand side is the *baroclinic* source obtained by taking the curl of Boussinesq buoyancy $f_b = \beta(T - T_{amb})\hat{y}$: horizontal temperature gradients at the plume edge inject the counter-rotating vortex pairs that roll rising columns into mushroom caps. This single term closes the temperature → buoyancy → vorticity loop that gives fire its characteristic motion (Nguyen et al. 2002; Selle et al. 2005).

Vorticity is carried by particles as circulation Γ_i. Per frame, each slice performs six steps on an $n_x \times n_y = 32 \times 48$ grid over a 2×3 world-unit domain:

1. *Temperature advection.* The grid temperature field T is advected semi-Lagrangianly (Stam 1999) with the previous frame's velocity, heated near the source by $T \mathrel{+}= \Delta t\, s_0 \exp(-\kappa_s \tilde{x}^2)$ in the source band, cooled by $T \mathrel{*}= (1 - 1/2\,\kappa_c \Delta t)$, and clamped.

2. *Baroclinic injection.* Each particle accumulates circulation from the local horizontal temperature gradient (centered differences of the advected field):

$$\Gamma_i \leftarrow (1 - \lambda_\Gamma \Delta t)\Gamma_i + \Delta t\, \beta \frac{\partial T}{\partial x}\Big|_{x_i}. \quad (2)$$

3. *P2G.* Particle circulations are deposited onto the grid vorticity ω with bilinear weights.

4. *Streamfunction solve.* We solve the Poisson equation $\nabla^2 \psi = -\omega$ with $K = 26$ Jacobi iterations and homogeneous Dirichlet boundaries.

5. *Velocity reconstruction.* $u = \nabla^\perp \psi = (\partial_y \psi, -\partial_x \psi)$ by centered differences. This step is solenoidal for *any* ψ, so truncating the Jacobi solve costs vorticity accuracy but cannot introduce divergence *here*; the buoyancy term added in step 6 can, and does (see below).

6. *G2P and advection.* Particles gather the grid velocity bilinearly, add direct buoyancy $\beta_b T_i \hat{y}$, a small divergence-free curl-noise perturbation (Bridson et al. 2007), wind, and the global nudge of §4.2, then advect with explicit Euler.

Note that buoyancy appears twice by design, with independent gains: the baroclinic term (Eq. 2, gain β) generates the *rotational* response—edge rolling and mushroom caps—while the direct term (gain β_b) is an additional control force for bulk rise. Decoupling the two is an artistic choice, not a derivation, and it has a consequence we state plainly: $\nabla \cdot (\beta_b T \hat{y}) = \beta_b \partial T / \partial y \neq 0$ in a thermal plume, so while the reconstructed field $\nabla^{\perp}\psi$ is solenoidal, the field that actually advects particles is not. With $\beta_b = 0.9$ against $\beta = 1.4$ this term is the same order as the baroclinic response. We therefore claim only that the solver needs no pressure *solve*—the property that buys the frame budget—and not that the advecting velocity is divergence-free.

Steps 3–5 constitute the vortex-in-cell velocity reconstruction (Cottet and Koumoutsakos 2000): they replace the $O(N_p^2)$ regularized Biot–Savart sum $u(x) = \sum_j \Gamma_j K_\varepsilon(x - x_j)$ (vortex-blob kernel $K_\varepsilon(r) = 1/2\pi(-r_y, r_x)/\|r\|^2 + \varepsilon^2$ (Chorin 1973; Beale and Majda 1985)) with an $O(N_p + G)$ transfer–solve– gather cycle, where G is the (small) grid size. At 1300 particles and a 32×48 grid per slice, nine slices leave ample headroom within a frame budget even in single-threaded JavaScript, whereas the naive Biot–Savart summation (retained in our codebase as a reference path) scales quadratically in the particle count.

Thermodynamics and lifecycle. Particles carry their own temperature, cooled nonlinearly, $dT_i/dt = -\kappa T_i^2$, which concentrates brightness at the hot base and produces the natural fast fade toward the flame tips; density decays exponentially. Particles are recycled when cold, exhausted, or out of bounds, and re-emitted at the source (emission balanced against the cull rate, up to a 1300-particle cap), so the particle count—and hence the Gaussian count—stays bounded.

4.2 Coarse Global Solver and Inter-Layer Coupling

Independent slices produce N *uncorrelated* fires. Production layered fire solved this with a splat kernel spanning neighboring slices (Horvath and Geiger 2009); we instead couple on the simulation side. A single coarse solver (grid 16×24, same structure as §4.1 but grid-only, with a quasi-static vorticity estimate $\omega = \beta \partial T / \partial x$ and $K = 30$ Jacobi iterations) is advanced once per frame to produce a global velocity field U representing the directable bulk motion of the whole fire. During each slice's G2P step, every particle is nudged toward the global field:

$$u_i \leftarrow u_i + c\left(U(x_i) - u_i\right), c \in [0, 1]. \quad (3)$$

$c = 0$ leaves slices fully independent (each layer visibly "its own fire"); $c = 1$ slaves all slices to the coarse solve (one coherent but overly smooth column); the default $c = 0.5$ retains per-slice turbulent detail on top of a shared bulk motion (Figure 7). This is the two-level decomposition of Horvath and Geiger (2009)—coarse directable motion plus per-slice refinement—with the coupling expressed as a first-class, continuously tunable simulation parameter rather than a fixed projection kernel. The coarse field also offers a single low-dimensional handle to which direction forces *can* be confined; our prototype currently applies wind and source motion per-particle as well.

Inter-slice vorticity coupling.

Equation 3 couples every slice to a shared *global* field, but it does not couple slices to their immediate *neighbors*: two adjacent slices can follow the same bulk motion while their turbulent detail stays uncorrelated, and stacked-2D dynamics structurally drop the out-of-plane transport $\partial^2\omega/\partial z^2$ that would tie them together. We reintroduce this dropped coupling directly. After each slice deposits its particle circulation onto the grid vorticity ω_k (step 3), we add a z-Laplacian exchange with the adjacent slices' vorticity from the previous frame,

$$\omega_k \leftarrow \omega_k + \Delta t\,\kappa_z\left(\omega_{k-1}^{t-1} + \omega_{k+1}^{t-1} - 2\,\omega_k^{t-1}\right), \quad (4)$$

with zero-flux (Neumann) ends so that total circulation $\sum_k \omega_k$ is conserved. Production layered fire (Horvath and Geiger 2009) has no slice-to-slice communication at all—its slices are coupled only through the one-way coarse broadcast—so this term fills an obvious hole, but we are explicit about how modest it is: it is the standard three-point stencil for the term the model deleted, applied explicitly with a one-frame lag, and it carries no $1/\Delta z^2$, so κ_z is a per-frame exchange rate rather than a diffusivity and is not comparable across slice counts. It is *off* by default (Table 3), its controlled statistical effect was inconclusive (§7), and its spectral effect is smaller than that of the coarse coupling it complements. We include it because the mechanism is sound and the hole is real, not because we can show it matters. The two mechanisms are complementary by design: Eq. 3 imposes global bulk coherence, Eq. 4 exchanges local neighbor detail. Quantifying the operator's statistical effect proved subtle—our controlled measurement in §7 is a deliberate negative result about the obvious probe. Crucially, Eq. 4 models the *coupling* that stacking drops; it does *not* recover true 3D vortex stretching $(\omega \cdot \nabla)u$, whose spectral consequences we treat honestly in §8.

4.3 Slice Geometry

Layer $k \in \{0, \ldots, N-1\}$ is assigned the depth offset $z_k = (k/(N-1) - 1/2)\,D$ along the stack axis, with a mild per-layer opacity fade $1 - 0.3|z_k|$. We support two slice frames:

View-aligned (default).

The in-plane axes are the camera's horizontal right vector and the world up vector, and layers are offset along the horizontal camera-forward direction. Slices re-orient continuously as the camera orbits, so the stack always presents its face—the choice of Horvath and Geiger (2009), without their per-view re-simulation, because our simulation state lives on particles rather than on slice rasters.

World-fixed.

Slices are pinned to a world plane and offset along the world normal. Orbiting reveals the stack edge-on and the volume collapses into stripes (Figure 6).

Viewed edge-on, a world-fixed stack degrades into visible stripes while the view-aligned stack continues to read as a volume (§7). Because re-orientation only changes the *lift* of in-plane particle coordinates into world space—not the simulation state—view-aligned slicing costs nothing and introduces no re-simulation popping; the known failure modes of view-locked slices

under fast camera motion (Horvath and Geiger 2009) are correspondingly milder but not eliminated (§8).

5 Emissive Gaussian Rendering

5.1 Particle-to-Gaussian Mapping

Table 2. *The complete simulation-to-rendering mapping. Every attribute of the rendered Gaussian is read directly off the simulation particle; there is no intermediate representation.*

particle attribute	Gaussian attribute	mechanism
position x_i + layer k	mean μ_i	slice-frame lift (§4.3)
temperature T_i	color c_i	blackbody ramp (§5.4)
density d_i	opacity α_i	§5.3
velocity gradient L_i	covariance Σ_i	Eq. 5
trajectory history	filament axis	Eq. 6

Table 2 lists the mapping. Each particle is instanced as a camera-facing quad whose vertex shader expands the unit square along the Gaussian's major and minor semi-axes and projects with the full perspective camera; the fragment shader evaluates the Gaussian footprint $g = \exp(-3r^2)$ modulated by a four-octave value-noise FBM detail factor that breaks the characteristic "blobby" look of isolated splats. Instance data is a flat 14-float record (position, two axes, fade, T, d, seed, type)—the direct render-side realization of the simulation contract.

5.2 Flow-Driven Covariance

Isotropic splats read as bubbles; real flames are sheared into streaks and wisps by the flow. Analogously to how PhysGaussian transforms covariances by the deformation gradient (Xie et al. 2024), we evolve each particle's in-plane 2×2 covariance directly by the local velocity gradient $L = \nabla u$ (measured from the slice grid by centered differences):

$$\dot{\Sigma} = L\,\Sigma + \Sigma\,L^{\top}, \quad (5)$$

integrated explicitly with gain k_{Σ}, relaxed toward identity at rate k_r, and stabilized in closed form: after a 2×2 eigen-decomposition we renormalize $\det \Sigma = 1$ and clamp the leading eigenvalue $\lambda_0 \le \kappa_{max}$ (with $\lambda_1 = 1/\lambda_0$ from the determinant constraint, this bounds the splat's semi-axis ratio a/b at κ_{max}). Under $\mathrm{tr}\, L = 0$ exact integration would preserve $\det \Sigma$; the renormalization removes explicit-integration drift. Equation 5 is the differential form of PhysGaussian's $\Sigma = F\,\Sigma_0\,F^{\top}$ (Xie et al. 2024) —but for a gas, F and Σ_0 are ill-defined absent a rest configuration, which makes the incremental form the natural one; and because the clamps act on the rendered quantity itself, the unbounded-elongation ("sliver") failure mode reported for Gaussians under sustained shear (Xie et al. 2024; Feng et al. 2025) cannot occur. The splat's semi-axes are then $a = s\sqrt{\lambda_0}$, $b = s\sqrt{\lambda_1}$ along the eigenvectors, so vortical shear visibly stretches blobs into licks of flame (Figure 8). Attaching per-particle anisotropy to a coarse solve echoes anisotropic

turbulence particles (Pfaff et al. 2010), with the anisotropy here doubling as the render primitive's footprint.

As an optional third tier, a trajectory *filament* stretch elongates each Gaussian along its recent path using an exponentially lagged shadow position p^{lag}:

$$p^{lag} \leftarrow p^{lag} + \eta\,(p - p^{lag}),\; a = s + k_f \left\| p - p^{lag} \right\|, \quad (6)$$

turning the particle set into short path-integrated streaks reminiscent of vortex filaments (Angelidis and Neyret 2005; Weißmann and Pinkall 2010)—an instantaneous-shear complement particularly effective for embers and fast plumes.

5.3 Unified Sorted Compositing

All dynamic and procedural-scene Gaussians—flame, smoke, embers, and the 640-splat background environment—are gathered into one buffer, sorted front-to-back by view depth, and rasterized in a single instanced draw into an RGBA16F target cleared to zero coverage. Fragments output premultiplied $(c_i \alpha_i, \alpha_i)$ and are blended with

$$C \leftarrow C + (1 - A)\, c_i \alpha_i,\; A \leftarrow A + (1 - A)\, \alpha_i, \quad (7)$$

(GL blend `ONE_MINUS_DST_ALPHA, ONE`)—exactly the front-to-back emission–absorption compositing of the 3DGS forward pass (Kerbl et al. 2023; Zwicker et al. 2001). The two media differ only in their opacity models: fire is optically thin and emission-dominated, so its Gaussians carry low α ($\alpha_i = clamp\big((\alpha_0 + \alpha_1 d_i)\, T_i D_i, 0, \alpha_{max}\big)$ with small α_{max}), accumulating radiance while barely occluding; smoke is absorption-dominated with high α and a density-modulated gray albedo warmed by residual temperature. Since emission and absorption are independent per-Gaussian channels, the *same* particle state spans the fire-to-smoke range under two scalar knobs (emission gain, absorption albedo) with no change to geometry, sort, or solver—consistent with Kirchhoff's law, hot particles emit and cooled ones absorb—so a single plume can transition from emissive flame at its hot base to absorptive smoke at its cooled tips. Because occlusion is resolved per primitive by the shared sort, the fire correctly sits *inside* the scene: environment splats in front of the flame occlude it, and the flame occludes those behind (Figure 1, middle)—in contrast to compositing a ray-marched volume over the scene as a whole (Shen et al. 2026). Procedural environment splats share the sorted buffer directly; large *loaded* reconstructions are merged into the same global depth order by bucketed interleaving (§5.5, Figure 5).

5.4 Blackbody HDR Shading and Post-Processing

Fire emission follows incandescence: we map particle temperature through a hand-tuned five-stop ramp approximating Planck blackbody colors (Nguyen et al. 2002; Pegoraro and Parker 2006; Horvath and Geiger 2009) (dark ember red → red → orange → yellow → near-white), scaled by an HDR emissive gain and a low-frequency flicker. Radiance accumulates far beyond display range, so the composited scene passes through a standard HDR chain: bright-pass extraction, five iterations of half-resolution separable Gaussian blur (bloom), then ACES filmic tone mapping (Narkowicz 2016), gamma, vignette, and a $1/255$ dither. Combined with the nonlinear particle cooling of §4.1, this reproduces the canonical luminance structure of real

flames—saturated white-hot core, graded orange body, dark red tips—without any texture assets (Figure 10).

5.5 Scene Composition and Secondary Effects

The environment is itself Gaussian content: either a procedurally scattered dark scene (640 flattened ground and enclosure splats) or a reconstructed 3DGS capture loaded from a standard Inria `.ply` (scales exponentiated, opacities sigmoided, DC spherical-harmonic color; subsampled to a budget of up to 120k splats). Procedural environment splats join the sorted buffer of §5.3 directly, giving per-primitive mutual occlusion with the fire. Loaded reconstructions are merged into the same global depth order by *depth bucketing*: each frame, scene splats are counting-sorted into $K = 96$ view-depth buckets in $O(n + K)$, and the renderer walks the buckets near-to-far, interleaving ranges of the (exactly sorted) dynamic stream with ranges of the bucketed scene stream under the shared blend state, coalescing adjacent same-stream ranges to minimize state changes. Order is thus exact within the dynamic stream and bucket-granular between and within the streams—strictly finer than the per-slice compositing granularity of layered production fire (Horvath and Geiger 2009). The bucketing adds no measurable cost: with the interleave on vs. a naive draw-after pass we measure 3.5 vs. 3.6 ms per frame at 3.2k scene splats and 13.0 vs. 13.3 ms at 79k splats (informal, development laptop; Figure 5).

In-scene ignition. Because the fire shares the scene's depth order rather than sitting in front of a backdrop, a source can be lit *on a reconstructed surface*: the plume then rises inside the captured geometry and is occluded per primitive by the objects around it (Figure 5, left). This in-place authoring is structurally unavailable to systems that composite fire over an opaque scene (Shen et al. 2026). Ignition composes with the fire-spread loop of §5.6, which turns a single lit point into scene-wide, multi-source fire.

The fire acts as a light source on the scene through an inexpensive screen-space model: environment splats are warmed by a radial falloff around the flame's screen position with a floor-proximity boost—a deliberately cheap stand-in for volumetric light transport that nevertheless anchors the flame in the scene (§8). Sparse ember particles (velocity-aligned, twinkling Gaussians in the shared sorted pass), plus two screen-space effects—a mirrored planar ground reflection and an FBM heat-haze refraction above the flame—complete the presentation; all are optional.

5.6 Fire Spread on the Scene's Own Primitives

Because the environment is itself a set of point primitives, combustion state can live *directly on the reconstruction*: each scene splat carries heat h, fuel f, a burning/charred state, and a glow amount. Adjacency is a spatial hash built once in the reconstruction's source frame (invariant to scene placement), and per-frame work is restricted to the fire front— burning splats and their neighbors—capped at a fixed budget, plus an $O(n)$ heat-decay scan.

The dynamics close a loop between the simulation and the scene. *Scene ignition:* sampled hot flame particles deposit heat into nearby splats (h decays, so ignition requires sustained heating); past a threshold a flammable splat ignites. *Spread:* burning splats heat unburnt neighbors with a buoyant upward bias, following the standard heat-driven spread models of interactive wood combustion and mesoscale wildfire (Pirk et al. 2017; Hädrich et al. 2021)—the spread physics is

deliberately conventional; per-splat fuel supports non-flammable regions. *Consumption:* fuel depletes over a burn time τ; while burning, an HDR ember-glow term is added in the scene shader, and a char fraction drives the splat's albedo toward black and reduces its opacity—combustion is written directly into the rendered attributes of the captured primitives. *Scene-to-simulation:* burning splats inject temperature into the nearest slice's T field and spawn flame particles there (round-robin, budgeted), so ignited geometry grows its *own* plumes through the full baroclinic loop.

Figure 3 shows the loop end to end on the test scene: a single ignition inside the fire ring spreads through the fuel, the burning splats glow in HDR and darken as they char, and—because they inject temperature and particles into the nearest slice—the plume above the fire is fed by the *scene*, not only by the original emitter. With a $\sim$190-splat active fire line the frame stays comfortably real-time (render 4.6 vs. 3.9 ms with the fire line active vs. quiescent, same machine as Table 4). Whether ignition can cross a gap wider than the adjacency radius depends on the plume reaching the far object, and therefore on scene scale relative to the slice-stack depth; we observed such a crossing in an earlier, more compact test configuration but do not claim it as a general property here.

We scope the claim precisely: heat-threshold spread with fuel and charring is long established on meshes and voxel grids (Pirk et al. 2017; Hädrich et al. 2021), and FieryGS spreads fire between contacting objects by solid heat diffusion on its occupancy grid, offline (Shen et al. 2026). What is new here is the *substrate and the loop*: combustion state carried on the splats of a captured scene, updated in real time, where the forward-simulated flame both ignites the reconstruction and is re-seeded by it. One honest caveat: the loop projects between world space and slice coordinates through the current slice frame, so under view-aligned slicing with a moving camera the spread coupling inherits the view-adaptation of §4.3; the world-fixed mode (or freezing the frame during spread) removes this dependence (§8).

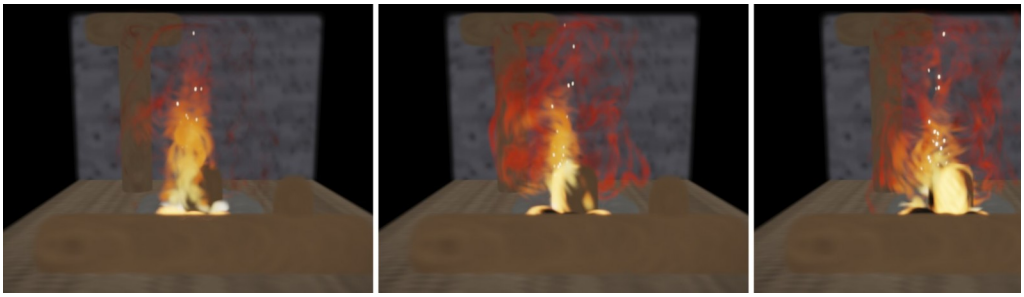

Figure 3. *Fire spread on the scene's own primitives (§5.6), one continuous simulation on the campfire test scene (plank floor, stone wall, stone fire ring, rear post, and a fallen log across the front). Left ($t \approx 3$ s): a single ignition inside the ring. Middle ($t \approx 10$ s): the fuel is burning—the splats themselves glow in HDR and inject temperature and particles into the slice solvers, so the plume above is fed by the scene. Right ($t \approx 22$ s): the fuel is consumed, with char written into the splats' albedo and opacity. The floor is marked non-flammable (zero fuel).*

6 Implementation

The entire system—simulation, sorting, rendering, UI—is a single dependency-free HTML file: simulation and per-frame depth sort run in JavaScript on the CPU; rendering is WebGL2 with instanced quads, an RGBA16F HDR target (8-bit fallback), and hand-written camera matrices. We emphasize what this implies: the method needs no GPU compute, no CUDA, and no preprocessing, and still reaches real-time rates (§7). Default configuration: $N = 9$ layers × 600 particles ($\sim$5.4k flame Gaussians), 640 scene splats, $\leq$160 embers—about 6.2k sorted Gaussians per frame, plus up to 120k additional splats when a reconstructed scene is loaded (merged into the global depth order by $O(n)$ bucketing; §5.5). Table 3 lists the main parameters. A headless harness drives the demo without a display and reads back probe images for the quantitative measurements in §7. For reproducibility, every stochastic simulation path (emission jitter, ember spawning, initial conditions) draws from a single seeded PRNG stream; combined with a fixed Δt, a run is fully determined by `(seed, steps)`, and an FNV-1a hash over the complete simulation state verifies bit-exact replay (§7).

Table 3. *Key parameters (defaults). World domain* 2×3 *units.*

symbol	meaning	default	section
N	slice layers	9	§4.3
N_p	particles per layer	1300	§4.1
$n_x \times n_y$	slice grid	32×48	§4.1
—	coarse grid	16×24	§4.2
K	Jacobi iterations	26 / 30	§4.1
β	baroclinic gain	1.4	Eq. 2
β_b	direct buoyancy	0.9	§4.1
κ	cooling rate	1.2	§4.1
c	global (coarse) coupling	0.5	Eq. 3
κ_z	inter-slice coupling	0 (opt.)	Eq. 4
D	stack depth	0.7	§4.3
k_Σ	covariance gain	1.3	Eq. 5
κ_{max}	eigenvalue clamp (bounds axis ratio)	3.6	Eq. 5
—	emissive gain / bloom / threshold	1.4 / 1.0 / 0.95	§5.4

7 Results and Evaluation

Deterministic replay.

All measurements below are taken through a deterministic-replay harness: every stochastic simulation path draws from one seeded PRNG stream, the solver is stepped at a fixed $\Delta t = 1/60$ s, and an FNV-1a hash over the complete particle, ember, and coarse-field state fingerprints each frame. Two runs from the same seed produce bit-identical state hashes after 120 steps; a different seed diverges—so every number in this section is exactly reproducible from a seed and a step count.

Performance.

Table 4 reports per-frame timings, measured over 240 simulated and 60 rendered frames per configuration at 1280×800 on a workstation (AMD Ryzen Threadripper PRO 3945WX; NVIDIA GeForce RTX 3080 via Chrome/ANGLE D3D11; simulation single-threaded in CPU JavaScript). The default configuration ($N = 9$, $N_p = 1300$; 12.4k sorted Gaussians) takes 17.8 ms per frame (56 FPS): 7.9 ms simulation, 4.1 ms gather-and-sort, 5.8 ms GPU-synchronized rendering. Simulation cost is close to linear in slice count—a least-squares fit gives $t_{sim}(N) \approx 0.36 + 0.87\,N$ ms (predicting 8.2 ms at $N = 9$ vs. 7.9 measured)—so $N = 17$ slices (22k particles) still sustain 29 FPS, while a lighter $N_p = 600$ preset runs at 92 FPS. We choose the $N_p = 1300$ default deliberately: below roughly one particle per solver cell the splat population is too sparse to read as a continuous flame, and image quality, not frame time, is what sets this budget. The same harness times the retained naive Biot–Savart reconstruction at matched counts: 1.83 ms for a *single* 600-particle slice (nine slices: ~ 16.5 ms for velocity reconstruction alone, $3.4\times$ our entire nine-slice simulation step) growing to 31.7 ms per slice at 2400 particles—the quadratic wall the $O(N_p + G)$ vortex-in-cell transfer–solve–gather cycle removes. For calibration of the design point (not of solver efficiency—these systems integrate far larger simulation state), the closest prior systems report 2.37 s per frame (FieryGS (Shen et al. 2026), RTX 4090D) and 38–228 s per *time step* (Gaussian Fluids (Xing et al. 2025), RTX 4090); production layered fire required ~ 20 s per slice per frame at $\sim 2048^2$ slice resolution on a GPU farm (Horvath and Geiger 2009), versus our 32×48—much of the gap is resolution deliberately traded away, the rest is architecture.

Table 4. *Per-frame cost (ms) vs. slice count N and particles per layer N_p. sim = solver step; sort = gather + exact depth sort; render = GPU-synchronized draw + HDR post. Workstation, single-threaded CPU JavaScript simulation; every row reproducible from seed 1.*

N	N_p	Gaussians	sim	sort	render	total (FPS)
1	1300	2.0k	0.98	0.72	1.53	3.2 (310)
3	1300	4.6k	3.20	1.84	2.11	7.2 (140)
5	1300	7.2k	5.75	2.90	3.36	12.0 (83)
9	**1300**	**12.4k**	**7.92**	**4.09**	**5.84**	**17.8 (56)**

N	N_p	Gaussians	sim	sort	render	total (FPS)
13	1300	17.6k	11.58	6.17	6.86	24.6 (41)
17	1300	22.8k	14.89	8.76	10.74	34.4 (29)
9	300	3.4k	3.60	0.92	1.85	6.4 (157)
9	600	6.1k	4.77	1.77	4.33	10.9 (92)
9	2200	20.2k	11.65	7.65	8.76	28.1 (36)

Figure 4. *Source hot-swap through the layer contract, same renderer and parameters. Left: all nine layers driven by the kinematic curl-noise plume—adequate bulk shape, but uniform ascent without vortical rolling. Right: all layers driven by the vortex-in-cell solver—baroclinic vorticity rolls the plume edges and produces the characteristic unsteady flame silhouette. Layers can be swapped per-layer at runtime with zero renderer changes.*

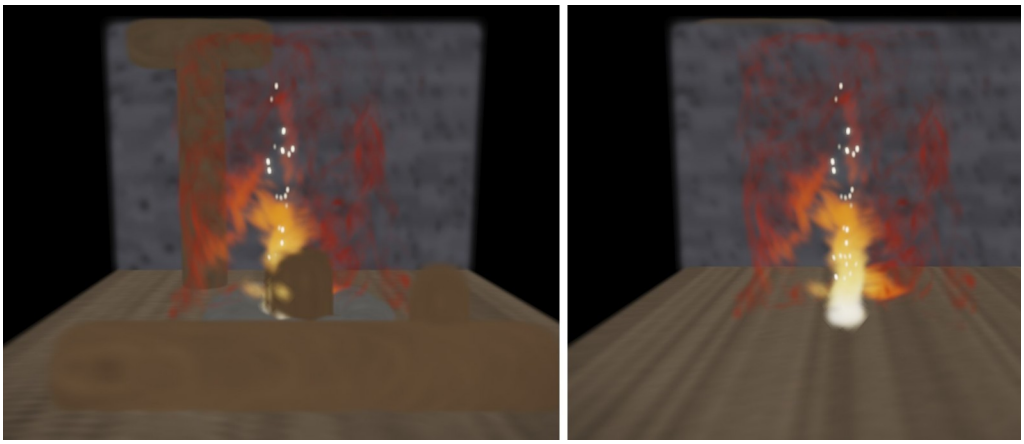

Figure 5. *Per-primitive occlusion with a loaded 3DGS scene (synthetic test reconstruction: plank floor, stone wall, a stone fire ring, a rear post, and a fallen log lying across the front of the fire;* 46*k splats). Left: bucketed depth interleaving (§5.5): the fallen log correctly occludes the base of the flame, the fire ring and fuel occlude it in the middle, and the flame occludes the rear post and wall. Right: naive draw-after compositing (prior behavior, and the analogue of backdrop compositing (Shen et al. 2026)): the flame renders in front of everything, and the log, the fire ring, the fuel and the rear post vanish entirely—without an intra-scene sort, the floor and*

wall drawn earlier exhaust the alpha budget of the pixels that physically belong to them. Same simulation state in both images.

Figure 6. *Slice orientation, viewed from the side (90° orbit). Left: view-aligned slices re-orient with the camera; the volume reads correctly from any azimuth. Right: world-fixed slices seen edge-on collapse into stripes (peak the failure mode that motivates view alignment.*

Figure 7. *Inter-layer coupling (Eq. 3). Left: $c = 0$, independent slices—the stack reads as several superimposed fires with uncorrelated sway. Right: $c = 1$, slices slaved to the coarse solver—one coherent but overly smooth column. The default $c = 0.5$ (Figure 1) preserves per-slice turbulence on a shared bulk motion. Snapshot field-correlation probes proved statistically unstable for quantifying either coupling mechanism (§7); the comparison here is deliberately qualitative.*

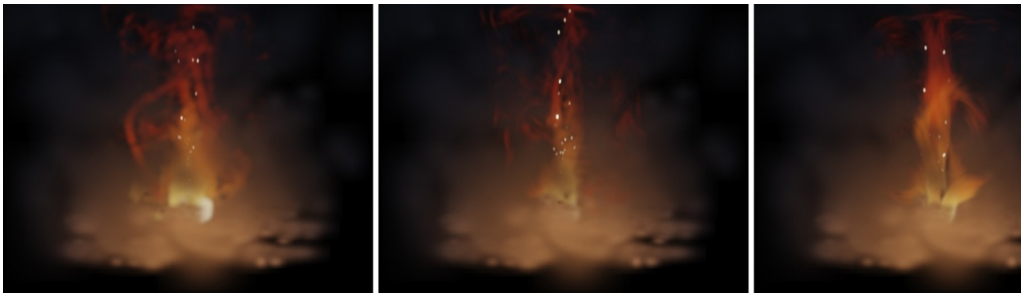

Figure 8. *Covariance dynamics. Left: isotropic splats ($k_\Sigma = 0$) read as soft blobs. Middle: flow-driven covariance (Eq. 5) shears splats into flame licks. Right: adding the trajectory filament stretch (Eq. 6) elongates fast particles into path-aligned streaks.*

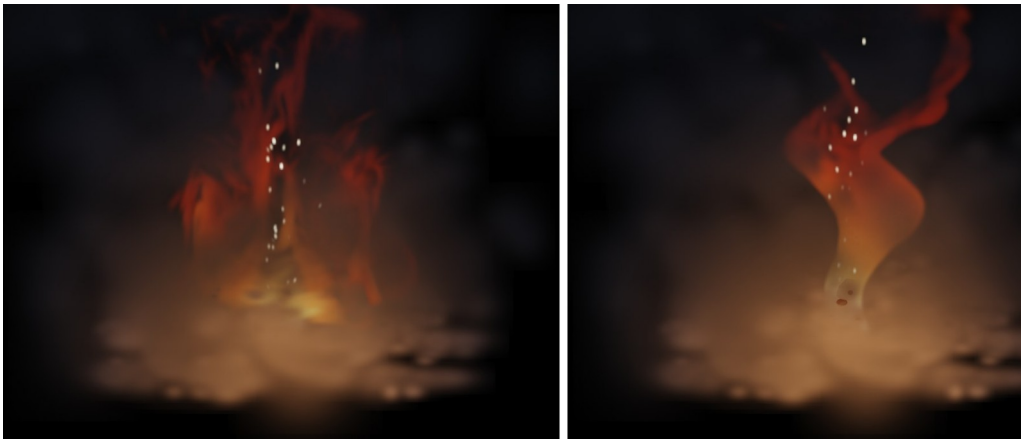

Figure 9. *Layered 2.5D vs. a matched-budget kinematic 3D mode. Left: nine coupled 2D VIC slices (default). Right: the same total particle budget advected by analytic 3D curl noise with entrainment—a volumetric alternative that lacks the baroclinic rolling produced by the slice solvers. The quantitative comparison against a genuine 3D reference solve is Figure 11; this panel is the qualitative matched-budget rendering ablation.*

Figure 10. *HDR post ablation: without (left) and with (right) bloom before ACES tone mapping. Bloom carries the perceived incandescence of the emission-dominated core.*

Slice orientation.

Orbiting the camera to 90° azimuth collapses world-fixed slices into visible stripes while view-aligned slices continue to read as a volume (Figure 6). We previously reported a peak-brightness probe here and have withdrawn it: at the default exposure the flame core is saturated at 758 of a 765 ceiling, so the statistic is clipped and separates the two modes only incidentally. The claim rests on the imagery and on the argument inherited from Horvath and Geiger (2009)—that a view-adapted decomposition avoids edge-on degeneracy—which our Lagrangian state gets at zero re-simulation cost. A non-saturating measure (silhouette coverage, or unclipped HDR

energy before tone mapping, swept densely in azimuth) is the right instrument and is future work.

Inter-slice coupling: a cautionary measurement.

We attempted to quantify the effect of Eq. 4 with a snapshot probe—the mean Pearson correlation of adjacent slices' velocity fields. An informal sequential measurement (settings changed on a warm, running simulation) suggested a strong monotonic rise with κ_z; under the deterministic-replay protocol (per setting: 3 seeds × 4 snapshots after a 600-frame cold-start warm-up, $c = 0$) that trend *does not replicate*: the probe sits at 0.26–0.38 across $\kappa_z \in [0, 1.2]$ with no significant trend, because the shared source geometry already induces strong structural correlation between slices and the probe's between-setting variance swamps the operator's marginal effect. We therefore make no quantitative coherence claim for the operator here. What survives is what is true by construction—the exchange is bidirectional, circulation-conserving, and the only slice-to-slice communication in a layered fire architecture—and the qualitative observation that concentrated vortices span several slices with it enabled. Whether the coupled stack is statistically closer to 3D is a question about spectra, which we answer next. We report this negative result deliberately: the replay harness exists so that informal numbers cannot survive into print.

Spectral comparison against a true-3D reference.

Figure 11 confronts the layered approximation with a matched true-3D reference: a $32 \times 48 \times 32$ Boussinesq Euler solve (semi-Lagrangian advection, Jacobi pressure projection, seeded symmetry-breaking forcing at the source; solver in supplemental) with the same domain, cell size, heat source, cooling, and Δt as the slice stack, compared on the same observable—the in-plane (u, v) fluctuation field on x–y planes sampled at the nine slices' exact world depths. The stack is exported from the shipping solver with procedural turbulence and wind disabled, isolating solver dynamics; both sides use the replay protocol (40 snapshots after warm-up, seeded). Three findings. *(i) The spectral gap is large and has the predicted sign:* the 3D reference distributes energy broadly (48 % in the largest-scale bin; $\sim$29 % in the mid-band $k \in [2, 4]$ cycles/unit, tracking $k^{-5/3}$), while every stack configuration concentrates 83–92 % of its energy in the largest-scale bin and carries only 2–5 % in the mid-band. We are careful about what this does and does not establish. It is consistent with the missing vortex-stretching cascade (§8), but that attribution is not isolated: this export ran at 600 particles per layer—0.39 per solver cell, well under the 4–8 that particle–grid practice assumes—with $K = 26$ Jacobi sweeps on a 48-cell axis and explicit Euler advection, and each of those is itself a low-pass filter on the small scales. What the measurement licenses is a *budget*: the layered stack, as configured here, sits this far from a matched 3D solve. Separating the dimensional cause from the discretization causes requires the refinement study in particles-per-cell, K, and Δt that we have not yet run. *(ii) Coupling nudges the spectrum toward 3D, but modestly:* the log-spectral distance to the reference falls from 0.836 (uncoupled) to 0.810 (κ_z only) to 0.779 (coarse coupling)—the two-level coupling does more for spectral shape than the inter-slice exchange, and neither closes the gap. *(iii) 3D flow is z-contiguous in a way the stack is not:* adjacent reference planes correlate at $\rho \approx 0.93$, adjacent slices at only 0.13–0.19 (κ_z raising it slightly and consistently). These curves make §8's honesty quantitative—and they are the explicit target any future inter-slice coupling

must be measured against. The reference itself is a modest solve ($32 \times 48 \times 32$, Jacobi projection); we use it as a matched control, not as a converged ground truth, and say so.

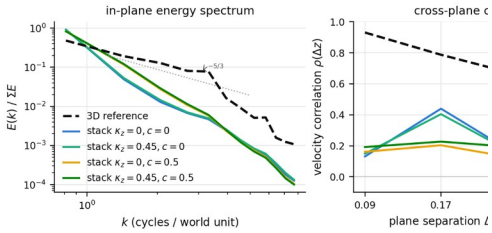

Figure 11. *The layered approximation, measured against a matched true-3D Boussinesq reference (dashed). Left: normalized in-plane energy spectra. The 3D reference spreads energy across scales ($\sim k^{-5/3}$ through the mid-band); all stack configurations are far steeper, concentrating energy at the largest scales—the predicted consequence of dropping vortex stretching. Coupling (c, κ_z) moves the spectrum toward the reference only modestly. Right: cross-plane velocity correlation at the slices' depths. The 3D flow is strongly z-contiguous ($\rho \approx 0.93$ adjacent); the stack is not (0.13–0.19), with κ_z helping slightly. (The $\Delta z = 0.17$ bump is a seeding artifact: the per-layer seed constant places adjacent layers' initial circulation nearly in anti-phase and next-nearest layers nearly in phase.) Correlations shown for $\Delta z \leq 0.35$; larger separations in the reference are contaminated by the source's mirror symmetry (supplemental).*

Scene occlusion.

Figure 5 exercises the compositing claim directly: a loaded 3DGS scene with a fallen log between the camera and the flame. With bucketed depth interleaving the log occludes the flame per primitive; with the naive draw-after pass (equivalent in spirit to backdrop compositing (Shen et al. 2026)) the flame incorrectly renders in front of the entire scene—and the unsorted scene additionally destroys its own internal ordering. The interleave is free: 3.5 vs. 3.6 ms per frame at 3.2k scene splats, 13.0 vs. 13.3 ms at 79k (both within a 60 FPS budget; same machine as Table 4).

Ablations.

Figures 4–10 ablate the major components: simulated vs. procedural sources, coupling strength, covariance dynamics, the layered representation itself, and HDR post. Each toggle is live in the interactive prototype.

Interactive control.

Because dynamics are forward-simulated, the system responds immediately to user input: shift-dragging moves the fire source and the plume follows with vortical lag; wind tilts the column; the coupling, buoyancy, baroclinic, and cooling gains are all live sliders; and fire and smoke are two shading presets over the same particle state, switchable per keystroke.

Comparisons.

8 Limitations and Future Work

Physics of stacked 2D slices.

Vortex stretching $(\omega \cdot \nabla)u$ vanishes in 2D, and 2D turbulence cascades energy toward *large* scales, opposite to 3D; a slice stack therefore cannot reproduce the statistics of three-dimensional fire turbulence, however plausible it looks. We have now *measured* this gap (Figure 11): the stack concentrates 83–92 % of in-plane energy in the largest-scale bin where a matched 3D reference holds 48 %, and adjacent-plane coherence is 0.13–0.19 vs. the reference's 0.93. Our inter-slice coupling operator (Eq. 4) reintroduces the dropped out-of-plane *transport*, but it is a diffusive exchange—it does not supply the vortex-stretching *amplification* that drives the forward 3D cascade—and accordingly it narrows the measured spectral distance only modestly. We deliberately trade physical fidelity for interactivity in a regime—turbulent, shapeless, emission-dominated media—where the eye is forgiving: visual plausibility rests on the large scales the stack does capture plus appearance-side detail (FBM footprints, flow-driven covariance, bloom), not on reproducing 3D statistics. Figure 11 is the standing target for any future coupling design.

Illumination.

Fire-to-scene lighting is a screen-space approximation, not transport: Gaussian scene splats are warmed by proximity to the flame's image-space position. Correct volumetric emission from Gaussian mixtures is available in a ray-traced formulation (Condor et al. 2025); a hybrid that shades nearby scene splats from the flame's particle set (a few thousand point lights, amenable to clustering) is future work, as are cast shadows and smoke self-shadowing.

Renderer completeness.

Our rasterizer implements the forward 3DGS math (sort, footprint, emission–absorption blend) but not the differentiable backward pass—our demo has no reconstruction objective—nor view-dependent spherical-harmonic color, which isotropic incandescence does not need. The FBM modulation of the splat footprint also departs from the pure Gaussian kernel, so fire splats exported to a stock 3DGS viewer would render softer than shown here; an export-compatibility experiment would sharpen the GS-native claim. The exact CPU sort of the dynamic set is adequate at 10^4 Gaussians and the $O(n)$ scene bucketing scales to 10^5 (§5.5), but ordering between the streams is bucket-granular, and tile-binned GPU sorting remains the right implementation for dense fire inside full reconstructed scenes.

View dependence and multi-view consistency.

View-aligned slices make the fire's world-space geometry a function of the rendering camera: the same particle lifts to different world positions as the camera orbits, so two simultaneous viewers—or the two eyes of a stereo pair—observe slightly different fires. This is the sharpest tension in the "GS-native" framing: the fire's *attributes* are camera-independent, but its slice lift is view-adapted, unlike a static reconstruction. The world-fixed mode removes the dependence at the cost of edge-on collapse (Figure 6); for stereo, sharing one lift per eye pair is a cheap fix.

More fundamentally, because particles are world-space objects, slices can be re-binned to a new axis without restarting the simulation—a remedy unavailable to raster-state slice methods, which we have not yet implemented. Very fast camera motion likewise changes the decomposition axis mid-shot, and flows predominantly along the view axis remain the weakest case (Horvath and Geiger 2009).

Radiometric level of detail.

In the optically-thin, emission-dominated regime the front-to-back accumulation of Eq. 7 is approximately additive, so distant fire clusters could be merged into fewer Gaussians while conserving the summed emission $c\,\alpha$ and its emission-weighted second moment—a radiance-conserving LOD criterion, distinct from the photometric-loss LOD of opaque-splat hierarchies (Kerbl et al. 2024). This is the concrete route to dense fire inside large reconstructions; we have not implemented it.

Geometry-aware flow.

The loaded scene could also act on the *simulation*, not only the compositing: rasterizing a signed-distance field sampled from the 3DGS scene into each slice's streamfunction Poisson solve as a free-slip (constant-ψ) boundary would deflect the flame around the captured geometry in real time— the per-slice, Lagrangian counterpart of the occupancy-grid obstacle boundaries that offline scene-combustion imposes (Shen et al. 2026).

Toward reconstruction.

The mapping of Table 2 is differentiable end-to-end in principle. Because the forward model exposes only a handful of interpretable controls (source, wind, buoyancy, baroclinic and coupling gains, cooling), fitting *those* to a video summary statistic—a far lower-dimensional, better-constrained inverse than fitting a free field or free Gaussians (Nazarenus et al. 2025; Yu et al. 2023), and solvable even with gradient-free system identification (Jatavallabhula et al. 2021) —would recover not a frozen radiance field but a *running, editable* simulation. Demonstrating a single fitted example is the natural next step.

What this evaluation does not yet cover.

We state the gaps explicitly rather than leave them to be discovered. *(i) Real captures.* Every scene-side result—occlusion, in-scene ignition, spread—uses a synthetic four-object `.ply`, the most favorable possible input for both depth bucketing and a splat adjacency graph. Our `.ply` path also flattens each Gaussian to its two largest principal axes rather than projecting the full 3D covariance through the perspective Jacobian, so it is an approximation of the 3DGS forward pass, not an implementation of it; a correct EWA path validated against a reference rasterizer is a prerequisite for running real reconstructions, and both are in progress. *(ii) Baselines.* We report no head-to-head against another system. The comparison we consider most informative—a 3D solver rendered through *our own* pipeline at matched wall-clock, isolating the layered decomposition from every other difference—is the one we would run first. *(iii) Hardware.* All timings come from a single workstation; a low-end second point would bound the interactive claim. *(iv) Ablations.* A slice-count sweep at fixed total particle budget, and a non-saturating viewpoint-isotropy measure swept densely in azimuth, would replace the qualitative arguments

in §4.3. *(v) Perceptual evidence.* Our central trade—physical fidelity for interactivity in a regime where the eye is forgiving—is an empirical claim about viewers that we argue for but do not test.

9 Conclusion

We have presented a system in which fire and smoke live natively inside Gaussian-splatting scenes: not reconstructed, not ray-marched over a backdrop, but forward-simulated in real time by vortex particles that *are* the rendered Gaussians. Its most distinctive behavior is a closed loop —the flame ignites the reconstruction, combustion state lives on the scene's own splats, and burning geometry re-seeds the solver—in which fire crosses a gap through the simulated flow rather than through any adjacency structure. The enabling observation, that Lagrangian vortex elements and 3D Gaussians are the same class of point primitive, collapses the simulation-to-rendering pipeline to an attribute mapping; a solver that needs no pressure solve is what keeps that loop inside an interactive frame budget. We have also measured what the layered approximation costs against a matched 3D reference, and report that gap rather than claim it away. Having demonstrated the pattern across flame, smoke, and embers, we conjecture that the same particle-set unification extends to other optically thin, surfaceless Lagrangian phenomena (sparks, spray, dust, aurorae), and that GS-native effects are a necessary ingredient if reconstructed scenes are to become fully dynamic worlds.

A Per-Slice Solver Pseudocode

```
step_slice(layer L, dt):
  # 1. temperature (grid, semi-Lagrangian)
  for cell g:
      T'[g] = bilerp(T, x_g - dt*u_prev(x_g))
  heat source band
  cool T' *= (1 - 0.5*kc*dt); clamp
  # 2. baroclinic injection (particles)
  for particle i:
      dTdx = (T'(x_i+h/2) - T'(x_i-h/2)) / h
      G_i = (1-lam*dt)*G_i + dt*beta*dTdx
  # 3. P2G: splat G_i -> omega (bilinear)
  # 4. Poisson: lap(psi) = -omega,
  #    K Jacobi sweeps, psi = 0 on boundary
  # 5. u = (dpsi/dy, -dpsi/dx)  (centered)
  # 6. G2P + advect
  for particle i:
      v = bilerp(u, x_i)
      v.y += beta_b * T_i        # buoyancy
      v += c*(U_coarse(x_i) - v) # nudge
      v += turb*curl_noise(x_i,t)
      x_i += dt * v
      T_i -= dt * kappa * T_i^2  # cooling
      d_i *= (1 - 0.3*dt)
      recycle if cold/exhausted/out of bounds
```

B Supplemental Material

Video (95 s).

Six single-take sequences, unedited: (1) real-time capture at a fixed camera, with the live-measured frame time on screen; (2) a full 360° orbit at constant rate; (3) the *failure case*—a fast azimuth change, shown rather than cut around; (4) interaction: dragging the source, wind, and the fire/smoke re-shading of one particle state; (5) a coarse-coupling sweep $c : 0 \rightarrow 1$ at matched wall-clock from the same seed; and and (6) the fire-spread loop on a loaded scene, in which burning splats feed the solver and the plume above the fire is sustained by the scene. Frames are rendered deterministically at $\Delta t = 1/60$ s from a seed and encoded at 60 fps, so playback speed equals simulation speed; the frame-time overlay is the rate measured live on the same machine and configuration (18.4 ms, 54 FPS), which is consistent with Table 4.

Code.

The complete system is the single dependency-free HTML file released with this submission, together with the 3D reference solver and the spectral-analysis script used for Figure 11 and the scene generator used for the composition results.

References

Angelidis, Alexis, and Fabrice Neyret. 2005. “Simulation of Smoke Based on Vortex Filament Primitives.” *Proceedings of the 2005 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA ’05)*, 87–96. https://doi.org/10.1145/1073368.1073380.

Beale, J. Thomas, and Andrew Majda. 1985. “High Order Accurate Vortex Methods with Explicit Velocity Kernels.” *Journal of Computational Physics* 58 (2): 188–208. https://doi.org/10.1016/0021-9991(85)90176-7.

Bridson, Robert, Jim Hourihan, and Marcus Nordenstam. 2007. “Curl-Noise for Procedural Fluid Flow.” *ACM Transactions on Graphics* 26 (3): 46:1–3. https://doi.org/10.1145/1276377.1276435.

Brochu, Tyson, Todd Keeler, and Robert Bridson. 2012. “Linear-Time Smoke Animation with Vortex Sheet Meshes.” *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA ’12)*, 87–95.

Chorin, Alexandre Joel. 1973. “Numerical Study of Slightly Viscous Flow.” *Journal of Fluid Mechanics* 57 (4): 785–96. https://doi.org/10.1017/S0022112073002016.

Chu, Mengyu, Lingjie Liu, Quan Zheng, et al. 2022. “Physics Informed Neural Fields for Smoke Reconstruction with Sparse Data.” *ACM Transactions on Graphics* 41 (4): 119:1–14. https://doi.org/10.1145/3528223.3530169.

Condor, Jorge, Sébastien Speierer, Lukas Bode, et al. 2025. “Don’t Splat Your Gaussians: Volumetric Ray-Traced Primitives for Modeling and Rendering Scattering and Emissive Media.” *ACM Transactions on Graphics* 44 (1): 10:1–17. https://doi.org/10.1145/3711853.

Cottet, Georges-Henri, and Petros D. Koumoutsakos. 2000. *Vortex Methods: Theory and Practice*. Cambridge University Press.

Crane, Keenan, Ignacio Llamas, and Sarah Tariq. 2007. “Real-Time Simulation and Rendering of 3D Fluids.” Chap. 30 in *GPU Gems 3*, edited by Hubert Nguyen. Addison-Wesley.

Decaudin, Philippe, and Fabrice Neyret. 2009. “Volumetric Billboards.” *Computer Graphics Forum* 28 (8): 2079–89. https://doi.org/10.1111/j.1467-8659.2009.01354.x.

Fedkiw, Ronald, Jos Stam, and Henrik Wann Jensen. 2001. “Visual Simulation of Smoke.” *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH ’01)*, 15–22. https://doi.org/10.1145/383259.383260.

Feng, Yutao, Xiang Feng, Yintong Shang, et al. 2025. “Gaussian Splashing: Unified Particles for Versatile Motion Synthesis and Rendering.” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 518–29.

Gao, Yue, Hong-Xing Yu, Bo Zhu, and Jiajun Wu. 2025. “FluidNexus: 3D Fluid Reconstruction and Prediction from a Single Video.” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 26091–101.

Hädrich, Torsten, Daniel T. Banuti, Wojtek Palubicki, Sören Pirk, and Dominik L. Michels. 2021. “Fire in Paradise: Mesoscale Simulation of Wildfires.” *ACM Transactions on Graphics* 40 (4): 163:1–15. https://doi.org/10.1145/3450626.3459954.

Harris, Mark J. 2004. “Fast Fluid Dynamics Simulation on the GPU.” Chap. 38 in *GPU Gems*, edited by Randima Fernando. Addison-Wesley.

Horvath, Christopher, and Willi Geiger. 2009. “Directable, High-Resolution Simulation of Fire on the GPU.” *ACM Transactions on Graphics* 28 (3): 41:1–8. https://doi.org/10.1145/1531326.1531347.

Jatavallabhula, Krishna Murthy, Miles Macklin, Florian Golemo, et al. 2021. “gradSim: Differentiable Simulation for System Identification and Visuomotor Control.” *International Conference on Learning Representations (ICLR)*.

Jiang, Ying, Chang Yu, Tianyi Xie, et al. 2024. “VR-GS: A Physical Dynamics-Aware Interactive Gaussian Splatting System in Virtual Reality.” *ACM SIGGRAPH 2024 Conference Papers (SIGGRAPH ’24)*. https://doi.org/10.1145/3641519.3657448.

Kerbl, Bernhard, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. “3D Gaussian Splatting for Real-Time Radiance Field Rendering.” *ACM Transactions on Graphics* 42 (4): 139:1–14. https://doi.org/10.1145/3592433.

Kerbl, Bernhard, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. 2024. “A Hierarchical 3D Gaussian Representation for Real-Time Rendering of Very Large Datasets.” *ACM Transactions on Graphics* 43 (4): 62:1–15. https://doi.org/10.1145/3658160.

Luiten, Jonathon, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. 2024. “Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis.” *International Conference on 3D Vision (3DV)*.

Narkowicz, Krzysztof. 2016. *ACES Filmic Tone Mapping Curve*. Https://knarkowicz.wordpress.com/2016/01/06/aces-filmic-tone-mapping-curve/.

Nazarenus, Jakob, Simin Kou, Fang-Lue Zhang, et al. 2025. *Gaussians on Fire: High-Frequency Reconstruction of Flames*. arXiv:2511.22459.

Nguyen, Duc Quang, Ronald Fedkiw, and Henrik Wann Jensen. 2002. “Physically Based Modeling and Animation of Fire.” *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH ’02)*, 721–28. https://doi.org/10.1145/566570.566643.

Park, Sang Il, and Myoung Jun Kim. 2005. “Vortex Fluid for Gaseous Phenomena.” *Proceedings of the 2005 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA ’05)*, 261–70. https://doi.org/10.1145/1073368.1073406.

Pegoraro, Vincent, and Steven G. Parker. 2006. “Physically-Based Realistic Fire Rendering.” *Proceedings of the Second Eurographics Workshop on Natural Phenomena (NPH ’06)*, 51–59. https://doi.org/10.2312/NPH/NPH06/051-059.

Pfaff, Tobias, Nils Thuerey, Jonathan Cohen, Sarah Tariq, and Markus Gross. 2010. “Scalable Fluid Simulation Using Anisotropic Turbulence Particles.” *ACM Transactions on Graphics* 29 (6): 174:1–8. https://doi.org/10.1145/1882261.1866196.

Pfaff, Tobias, Nils Thuerey, and Markus Gross. 2012. “Lagrangian Vortex Sheets for Animating Fluids.” *ACM Transactions on Graphics* 31 (4): 112:1–8. https://doi.org/10.1145/2185520.2185608.

Pirk, Sören, Michał Jarząbek, Torsten Hädrich, Dominik L. Michels, and Wojciech Palubicki. 2017. “Interactive Wood Combustion for Botanical Tree Models.” *ACM Transactions on Graphics* 36 (6): 197:1–12. https://doi.org/10.1145/3130800.3130814.

Selle, Andrew, Nick Rasmussen, and Ronald Fedkiw. 2005. “A Vortex Particle Method for Smoke, Water and Explosions.” *ACM Transactions on Graphics* 24 (3): 910–14. https://doi.org/10.1145/1073204.1073282.

Shen, Qianfan, Ningxiao Tao, Qiyu Dai, et al. 2026. “FieryGS: In-the-Wild Fire Synthesis with Physics-Integrated Gaussian Splatting.” *International Conference on Learning Representations (ICLR)*.

Shui, Yunhao, Fuhong Zhang, Can Gao, et al. 2024. “FlameGS: Reconstruct Flame Light Field via Gaussian Splatting.” *Proc. SPIE 13652, International Conference on Signal Processing and Neural Network Applications (SPNNA 2024)*. https://doi.org/10.1117/12.3065237.

Stam, Jos. 1999. “Stable Fluids.” *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH ’99)*, 121–28. https://doi.org/10.1145/311535.311548.

Steinhoff, John, and David Underhill. 1994. “Modification of the Euler Equations for ‘Vorticity Confinement’: Application to the Computation of Interacting Vortex Rings.” *Physics of Fluids* 6 (8): 2738–44. https://doi.org/10.1063/1.868164.

Umenhoffer, Tamás, László Szirmay-Kalos, and Gábor Szijártó. 2006. “Spherical Billboards and Their Application to Rendering Explosions.” *Proceedings of Graphics Interface 2006*, 57–63.

Wang, Sinan, Yitong Deng, Molin Deng, et al. 2024. “An Eulerian Vortex Method on Flow Maps.” *ACM Transactions on Graphics* 43 (6): 248:1–14. https://doi.org/10.1145/3687996.

Weißmann, Steffen, and Ulrich Pinkall. 2010. “Filament-Based Smoke with Vortex Shedding and Variational Reconnection.” *ACM Transactions on Graphics* 29 (4): 115:1–12. https://doi.org/10.1145/1778765.1778852.

Wu, Guanjun, Taoran Yi, Jiemin Fang, et al. 2024. “4D Gaussian Splatting for Real-Time Dynamic Scene Rendering.” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 20310–20.

Xie, Tianyi, Zeshun Zong, Yuxing Qiu, et al. 2024. “PhysGaussian: Physics-Integrated 3D Gaussians for Generative Dynamics.” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 4389–98.

Xing, Jingrui, Bin Wang, Mengyu Chu, and Baoquan Chen. 2025. “Gaussian Fluids: A Grid-Free Fluid Solver Based on Gaussian Spatial Representation.” *ACM SIGGRAPH 2025 Conference Papers*. https://doi.org/10.1145/3721238.3730620.

Yu, Hong-Xing, Yang Zheng, Yuan Gao, Yitong Deng, Bo Zhu, and Jiajun Wu. 2023. “Inferring Hybrid Neural Fluid Fields from Videos.” *Advances in Neural Information Processing Systems (NeurIPS)*.

Yu, Zehao, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. 2024. “Mip-Splatting: Alias-Free 3D Gaussian Splatting.” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 19447–56.

Zhao, Zhiwei, Alan Zhao, Minchen Li, and Yixin Hu. 2025. *Vid2Fluid: 3D Dynamic Fluid Assets from Single-View Videos with Generative Gaussian Splatting*. arXiv:2503.00868.

Zhong, Licheng, Hong-Xing Yu, Jiajun Wu, and Yunzhu Li. 2024. “Reconstruction and Simulation of Elastic Objects with Spring-Mass 3D Gaussians.” *European Conference on Computer Vision (ECCV)*.

Zhu, Yongning, and Robert Bridson. 2005. “Animating Sand as a Fluid.” *ACM Transactions on Graphics* 24 (3): 965–72. https://doi.org/10.1145/1073204.1073298.

Zwicker, Matthias, Hanspeter Pfister, Jeroen van Baar, and Markus Gross. 2001. “EWA Volume Splatting.” *Proceedings of IEEE Visualization (VIS ’01)*, 29–36. https://doi.org/10.1109/VISUAL.2001.964490.